Архітектура системи

- **Presentation Layer:** Десктопний графічний інтерфейс користувача (WPF / Avalonia UI на базі .NET). Головне вікно побудоване на основі інтерактивної таблиці (DataGrid) із візуальною підсвіткою дедлайнів, панеллю фільтрації, діалоговими вікнами створення й редагування завдань та дашбордом персональної статистики.
- **Business Logic Layer:** Сервіси розподілу завдань, розрахунку динамічного пріоритету (на основі часу до дедлайну, коефіцієнта важливості та прогнозованої трудомісткості), валідації вхідних даних та алгоритмів агрегації статистики.
- **Persistence Layer:** Модуль збереження даних. На етапах 1–4 використовується файлове сховище (JSON через System.Text.Json) з автоматичною серіалізацією.
- **Integration Layer (з етапу 5):** Клієнтський модуль на базі HttpClient для взаємодії із зовнішніми REST API (Notion API / Google Classroom API, навчальний сервер авторизації), а також ORM Entity Framework Core для взаємодії з реляційною базою даних (SQLite / PostgreSQL).

Етапи реалізації проєкту

Етап 1. Базові сутності та базова бізнес-логіка (BLL)

- **Класи сутностей:**
  - TaskItem — завдання: унікальний ідентифікатор, назва, детальний опис, дата й час дедлайну, розрахунковий час на виконання (у хвилинах), статус («Нове», «В процесі», «Виконано», «Прострочено»), пріоритет.
  - Category / Project — проєкт або навчальна дисципліна (назва, кольорова мітка).
  - TaskPriority — перелік рівнів важливості (Низький, Середній, Високий, Критичний).
- **Функціонал бізнес-логіки:**
  - Реалізація базових CRUD-операцій: створення, редагування параметрів, видалення завдань та категорій.
  - Базова валідація даних: перевірка наявності назви, валідація дедлайну (заборона створення завдання з датою в минулому), перевірка допустимого діапазону оціночного часу.
  - Логіка зміни стану та позначення завдання як «Виконано».
  - Збереження та завантаження колекцій сутностей у файл JSON (без повноцінного інтерфейсу, демонстрація через юніт-тести або консольну точку входу).

Етап 2. Організація модуля десктопного інтерфейсу та розширення функціоналу

- **Організація Presentation Layer:**
  - Розробка головного вікна застосунку (MainWindow) з інтерактивною таблицею (DataGrid) для відображення списку завдань.
  - Створення діалогових вікон введення/редагування завдань з елементами вибору дати (DatePicker) та списками категорій і пріоритетів (ComboBox).
  - Налаштування прив'язки даних (Data Binding через ObservableCollection) для синхронізації змін у колекціях з інтерфейсом.
- **Збереження даних:** Автоматичне зчитування даних із файлу під час старту програми та автоматичний запис у файл після кожної операції додавання чи зміни.

- **Життєвий цикл завдань:** Фіксація часу завершення завдання, визначення факту прострочення дедлайну, заборона редагування для закритих завдань.
- **Пошук та фільтрація:** Текстовий пошук за назвою в реальному часі та панель комбінованих фільтрів (за категоріями, статусами, завданнями на сьогодні або на найближчі 3 дні).
- **Базовий розподіл:** Сортування таблиці за алгоритмом EDF (*Earliest Deadline First*) з урахуванням ваги пріоритету.

Етап 3. Аналітика, статистика та алгоритм «План на день»

- **Дашборд аналітики в інтерфейсі:**
  - Створення окремої вкладки/вікна зі статистикою: відсоток виконання вчасно (*On-Time Rate*), загальна кількість активних та прострочених завдань за обраний період (день, тиждень, місяць).
  - Зведена таблиця розподілу навантаження та кількості завдань за дисциплінами/категоріями.
- **Алгоритм «План на день» (Розумний розподільник):**
  - Введення користувачем доступного ліміту робочого часу на сьогодні (наприклад, 180 хвилин) у десктопному інтерфейсі.
  - Автоматичний відбір системою найбільш пріоритетних і термінових завдань, сумарний час яких не перевищує вказаний ліміт, із формуванням щоденного фокус-списку.
- **Збережені фільтри:** Збереження в інтерфейсі до 5 користувацьких пресетів фільтрації для швидкого перемикання подання таблиці.

Етап 4. Нові сутності, тайм-трекінг та балансування навантаження

- **Сутність WorkSession:** Фіксація фактично витраченого часу на конкретне завдання (вбудований у десктоп секундомір / таймер Pomodoro або ручне введення хвилин).
- **Сутність ExternalSourceInfo:** Метадані зовнішньої системи (платформа-джерело: Classroom, Notion; зовнішній ідентифікатор, посилання на задачу, стан синхронізації).
- **План-фактний аналіз:** Порівняння попередньо запланованої тривалості завдання з фактично витраченим часом за всіма сесіями WorkSession.
- **Детектор перевантаження (Burnout Detector):** Календарний аналіз щільності дедлайнів із кольоровим підсвічуванням днів із критичним навантаженням (коли сумарний плановий час перевищує допустиму добову норму).

Етап 5*. Інтеграція з віддаленими сервісами та перехід на базу даних

- **Аутентифікація та особистий кабінет:**
  - Інтеграція з навчальним сервером через HttpClient для реєстрації, входу та збереження облікових записів.
  - Вікно авторизації в десктопному клієнті, збереження сесійного токена, екран особистого кабінету користувача.
- **Зовнішній фетчинг завдань:**
  - Реалізація модуля для отримання завдань із зовнішніх сервісів (Notion API / Google Classroom API або Mock-сервера) через HTTP-запити.
  - Бізнес-логіка дедуплікації: перевірка наявності зовнішнього ID для виключення

дублікатів та оновлення дедлайнів, змінених викладачем/менеджером на вихідній платформі.

- **Перехід на Entity Framework Core:**
  - Заміна збереження у JSON-файли на реляційну базу даних (SQLite / PostgreSQL) через DbContext і міграції.
  - Збереження функції локального резервного експорту та імпорту в JSON-формат.

Етап 6*. Розділення ролей та координація через спільну БД

- **Режими інтерфейсу залежно від ролі:**
  - **Роль «Виконавець»:** Персональна таблиця завдань, запуск синхронізації з власними акаунтами Notion/Classroom, зміна статусів виконання, тайм-трекер та персональна статистика продуктивності.
  - **Роль «Куратор / Тімлід»:** Зведена матриця завантаженості команди чи академічної групи, призначення спільних завдань із фіксованими термінами, моніторинг академічних заборгованостей та списку прострочених робіт.
  - **Роль «Адміністратор»:** Керування обліковими записами, аудит синхронізацій, глобальні довідники категорій.
- **Координація через базу даних:**
  - Усі запущені екземпляри десктопного застосунку підключаються до спільної бази даних.
  - Реалізація повторного вичитування актуальних даних із БД під час перемикання екранів або за дією користувача для своєчасного відображення змін, внесених іншими ролями.